

TOS: An Actor-Oriented Blockchain Platform

Parallel Execution, Native Standard Library, and
Developer-First Design

Tomi Setsu

April 2026

Abstract

TOS is a high-performance blockchain platform designed from the ground up around the Actor Model. Every smart contract is a collection of fine-grained, independently-executing actors, giving TOS intra-contract parallelism that existing blockchains cannot achieve. The platform combines a Cell/BOC data layer—providing native Merkle proofs and compact serialization—with a simple key-value state interface exposed through a new Solidity-like language called Tola. TOS ships a native C++ standard actor library—analogueous to Ethereum’s OpenZeppelin but compiled into the node binary—enabling zero-deployment, zero-audit standard primitives for tokens, NFTs, governance, and DeFi.

Contents

1	Introduction and Motivation	3
1.1	The State of Blockchains in 2026	3
1.2	The Actor Model Insight	4
1.3	Design Principles	5
2	Architecture Overview	6
2.1	The Actor as the Atomic Unit	6
2.2	Parallel Execution	6
2.3	Cell Inside, KV Outside	7
3	Execution Model	7
3.1	Two Execution Paths	7
3.2	TVM Extensions	8
3.3	Transaction Execution	8
3.4	Execution Phases	9
4	The Tola Language	9
4.1	Design Goals	9
4.2	Actor Declaration	9
4.3	ActorId and Asynchronous Messaging	10
4.4	External Messages, Gas Control, and System Hooks	10
4.5	Inheritance and Code Reuse	11
4.6	Raw Cell Escape Hatch	12
5	Built-in Standard Actor Library	12
5.1	Why Built-in Standard Actors	12
5.2	Standard Actor Inventory	13
5.3	Usage Model	13
6	AI Agent Economy	14
6.1	Actors as Agent Wallets	14
6.2	Throughput for Machine-to-Machine Economies	15
6.3	Verifiable Agent Behavior	15
7	Privacy at the Actor Level	16
7.1	The Privacy Problem in Existing Blockchains	16
7.2	Actors as Natural Privacy Boundaries	16

7.3	ZK-Verified Execution: Verify, Don't Re-Execute	16
8	Post-Quantum Resilience	17
8.1	The Threat	17
8.2	Actor-Level Cryptographic Agility	17
8.3	Hash-Based State Integrity	18
9	Concrete Example: Parallel Token Transfers	18
10	Infrastructure Stack	19
11	Comparison with Existing Platforms	20
12	Roadmap	21
	Conclusion	21

1 Introduction and Motivation

1.1 The State of Blockchains in 2026

The blockchain landscape in 2026 is shaped by five converging forces:

1. **The AI agent economy.** Autonomous AI agents are becoming first-class economic actors. By early 2026, agents hold their own wallets, negotiate payments via standardized HTTP payment protocols [8], and execute thousands of micro-transactions per second without human intervention. Account abstraction and tool invocation standards (such as the Model Context Protocol) are maturing rapidly. This demands blockchains that can handle *massive volumes of small, autonomous, machine-to-machine transactions* with low latency and predictable fees—requirements that rollup-fragmented architectures and serial-execution VMs cannot meet.
2. **Throughput vs. decentralization.** High-throughput chains achieve speed by concentrating hardware requirements, while rollup-centric roadmaps fragment liquidity and user experience across hundreds of Layer-2 networks. Recent industry efforts toward parallel EVM execution and Verkle Tree state representations are promising, but remain incremental patches to fundamentally serial architectures designed a decade ago.
3. **Parallel execution vs. developer simplicity.** Object-based and resource-based parallel execution models have emerged, but their programming paradigms require developers to learn entirely new languages and mental models. Account-locking approaches require manual specification of all state touched by a transaction, creating friction and frequent failures. The vast majority of smart contract developers remain trained on Solidity-style syntax.
4. **Privacy as a first-class requirement.** Regulators demand transparency while users demand confidentiality. The zero-knowledge proof market is projected to reach \$7.6B by 2033. Industry efforts have produced encrypted rollup transactions, enterprise privacy sandboxes, and confidential compute frameworks for smart contracts. Yet no Layer-1 platform offers privacy as a *native, per-actor capability* rather than an afterthought bolted onto a transparent execution model.

5. **The quantum threat.** Recent estimates suggest a cryptographically-relevant quantum computer could break elliptic curve signatures—the scheme securing the vast majority of blockchains—in under nine minutes [10]. A significant fraction of on-chain value has exposed public keys and would be immediately vulnerable. NIST standardized post-quantum signature schemes (SLH-DSA/FIPS 205, ML-DSA/FIPS 204) in 2024, and the industry broadly targets 128-bit provable post-quantum security by decade’s end. Yet no shipping blockchain has a *modular, actor-level mechanism* for upgrading cryptographic primitives without a network-wide hard fork.

TOS addresses all five forces through a single architectural principle: the **Actor Model** applied at the smart contract level, with natural extension points for AI agent workloads, per-actor privacy, and post-quantum authentication.

1.2 The Actor Model Insight

Two axioms from first principles:

1. Two execution units can run in parallel *if and only if* their mutable state is completely disjoint.
2. Blockchain determinism requires all validators to agree on execution order and results.

The conventional account model treats each smart contract as a monolithic state blob. A popular token contract with millions of users must serialize *every* transfer because they all touch the same balance mapping. Sharding helps when different contracts are busy, but cannot help when one contract is the bottleneck.

The Actor Model solves this by decomposing a contract’s state into **independent actors**, each owning an isolated state partition with its own logical timeline. Actors within the same contract communicate via asynchronous messages, and—crucially—can execute **in parallel** because their state never overlaps.

Crucially, the actor boundary is also a natural **privacy boundary** and a natural **cryptographic boundary**: each actor can independently adopt encrypted state, zero-knowledge proofs, or post-quantum signature schemes without affecting other actors or requiring a global protocol upgrade.

1.3 Design Principles

TOS is governed by seven design principles:

1. **Actor as the minimum execution unit.** Not accounts, not transactions—actors. Each actor owns its state, processes one message at a time, and communicates only through async messages.
2. **Cell inside, KV outside.** Internally, TOS stores all state in an elegant Cell/Merkle data structure that provides native Merkle proofs and compact consensus encoding. Developers see only `state_get/state_put`—a key-value interface. Zero-cost abstraction: the compiler maps KV operations to Cell dictionary operations transparently.
3. **Native standard library.** The most-used contract patterns (tokens, NFTs, access control, governance, DeFi primitives) ship as native C++ built-in actors, compiled into the node binary. Users instantiate actors via message, not deployment. Audited once, shared by all.
4. **Familiar syntax.** The Tola language uses widely-adopted smart contract syntax conventions (`function`, `require`, `mapping`, `modifier`, `event`) so the largest developer community can onboard with minimal learning. The single new concept: cross-actor calls are `.send()`, not synchronous function calls.
5. **AI-agent ready.** Actors are the natural unit for agent wallets: each agent is an actor with its own state, spending policy, and message interface. The async messaging model matches the request/response pattern of MCP and x402, enabling high-throughput autonomous agent economies without blocking or reentrancy risks.
6. **Privacy-extensible.** Because each actor's state is an isolated `HashMapE`, a confidential actor can encrypt its state entries and prove correct execution via zero-knowledge proofs without touching any other actor's data. Privacy is a per-actor choice, not a network-wide mode switch.
7. **Quantum-resilient by design.** Actor-level authentication means each actor can independently adopt post-quantum signature schemes (e.g., SPHINCS+/SLH-DSA). A built-in `PostQuantumAuth` actor module can replace ECDSA for any actor without a chain-wide hard fork.

The upgrade is granular: critical actors (treasuries, bridges) can go post-quantum immediately while the rest of the network migrates at its own pace.

2 Architecture Overview

2.1 The Actor as the Atomic Unit

The minimum addressable unit in TOS is not an account but an **actor**. An account is simply a deployment container that holds shared code and groups related actors together:

```
Account (unique address)
+-- code: Cell          <- shared by all actors in this account
+-- balance: Coins      <- shared (account-level)
+-- actors: HashmapE     <- actor_id -> actor state (Cell dict)
|   +-- actor_0: HashmapE <- independent KV state
|   +-- actor_1: HashmapE <- independent KV state
|   +-- ...
+-- actor_lt: HashmapE   <- actor_id -> independent logical time
```

Each actor is identified by a three-part address: `workchain:account_id:actor_id`. The `actor_id` is deterministically derived: `actor_id = sha256(account_address||discriminator)`.

This design stands in contrast to conventional account-based chains where a single contract holds one monolithic state tree behind one logical timeline, forcing all operations to serialize.

2.2 Parallel Execution

Different actors under the same account execute in parallel because their state (`HashmapE`) is disjoint. The node's `td::actor` scheduler naturally handles this: each on-chain actor maps to a `td::actor::Actor` instance, and the scheduler's thread pool dispatches them concurrently. Same-actor messages remain serialized (`ActorLocker` guarantee).

	Account Model	Actor Model (TOS)
Minimum unit	Account	Actor
Parallelism	N accounts	$N \times M$ actors
Hot-contract throughput	$1 \times$	$M \times$ (actors per contract)
Reentrancy	Possible	Impossible by design

2.3 Cell Inside, KV Outside

TOS uses the Cell data structure as its internal storage layer. Every actor's state is a `HashMapE` (Cell-based dictionary). This yields three properties:

- **Native Merkle proofs.** The proof chain `block_root` $\rightarrow$ `shard_state` $\rightarrow$ `account` $\rightarrow$ `actor` $\rightarrow$ `key=value` is entirely Cell-based, giving every actor state entry a compact, verifiable Merkle path.
- **Uniform BOC encoding.** Blocks, transactions, and messages all use Cell encoding, providing a single serialization format across the entire stack.
- **Developers see KV.** The new TVM opcodes `STATEGET`/`STATESET`/`STATEDEL` wrap `HashMapE::get/set/remove`. Developers write `balance += amount`, and the compiler generates the Cell dictionary operations.

3 Execution Model

3.1 Two Execution Paths

TOS supports two execution paths sharing the same state interface (`vm::Dictionary` / `HashMapE`):

Path	For	Performance	Safety
Native C++	Built-in/system actors	Fastest	Trusted (ships with node)
TVM	User-deployed contracts	Medium	TVM sandbox

Built-in actors and user-deployed actors can exchange messages freely because they share the same state format and message protocol.

3.2 TVM Extensions

Four new TVM opcodes enable actor-native programming:

Opcode	Semantics
STATEGET	Read a key from the actor's HashmapE
STATESET	Write a key-value pair to the actor's HashmapE
STATEDEL	Delete a key from the actor's HashmapE
ACTORSEND	Send an asynchronous message to another actor

These opcodes are implemented on top of `vm::Dictionary` operations—no new data structures are needed. The lower-level `DICTGET`/`DICTSET` instructions also work directly on the actor's C4 register for advanced use cases.

3.3 Transaction Execution

The collator dispatches messages to actors via `td::actor::send_closure`. Each `OnChainActor` is a `td::actor::Actor` instance:

```
class OnChainActor : public td::actor::Actor {
    const block::Account* acc_;           // read-only:
    shared code
    vm::Dictionary actor_state_;          // exclusive:
    this actor's state
    LogicalTime actor_lt_;                // exclusive:
    this actor's timeline

    void execute(Ref<vm::Cell> msg,
                 td::Promise<Ref<vm::Cell>> promise) {
        auto result = impl_create_ordinary_transaction(
            msg, acc_, &actor_state_, &actor_lt_, ...);
        promise.set_result(std::move(result));
    }
};
```

The collator selects messages, enforces block limits, and assembles the block. Execution dispatch is asynchronous: the collator sends actor messages rather than invoking synchronous function calls.

3.4 Execution Phases

Each actor transaction follows a five-phase execution model:

1. **Storage phase** — compute and deduct storage fees
2. **Credit phase** — credit inbound message value
3. **Compute phase** — execute contract code (TVM or native)
4. **Action phase** — process output actions (messages, reserves)
5. **Bounce phase** — generate bounce message on failure

4 The Tola Language

4.1 Design Goals

Tola is a statically-typed, Solidity-like smart contract language that compiles to TVM bytecode. Its design goals are:

1. **Familiar** — reuses Solidity syntax conventions so developers can on-board in hours, not weeks.
2. **Actor-native** — `actor` replaces `contract`; `.send()` replaces synchronous `.call()`.
3. **Safe by default** — checked arithmetic, no reentrancy (impossible by design), no `delegatecall`, no `tx.origin`.
4. **100% FunC coverage** — every FunC primitive is accessible, either natively or through `unsafe cell {}` escape blocks.

4.2 Actor Declaration

```
actor TokenBalance {  
    uint64 balance;  
  
    function credit(uint64 amount) external {  
        balance += amount;  
    }  
}
```

```
    }

    function transfer(ActorId recipient, uint64 amount)
    external {
        require(balance >= amount, "insufficient");
        balance -= amount;
        recipient.send("credit", abi.encode(amount));
    }
}
```

Under the hood, `balance += amount` compiles to `STATEGET → ADD → STATESET` operating on the actor's `HashMapE`. The developer never sees `Cell`, `Slice`, or `Builder`.

4.3 ActorId and Asynchronous Messaging

`ActorId` is a 256-bit first-class type representing an actor's on-chain identity:

```
ActorId bob = ActorId.derive(self, "bob");
bob.send("credit", abi.encode(100));
// async: fire and forget, result comes back as a
// separate message
```

Send mode flags provide full control over message delivery, covering all of FunC's `send_raw_message` mode bits:

```
recipient.send("transfer", payload, SendOptions {
    value: 1000,
    mode: SendMode.PAY_FEES_SEPARATELY | SendMode.
        IGNORE_ERRORS,
    bounce: true
});
```

4.4 External Messages, Gas Control, and System Hooks

Tola covers the full FunC feature set:

- `receive_external` — handle wallet/dApp messages (FunC `recv_external`)
- `gas.accept()` — accept external messages and pay gas (FunC `accept_message`)

- `gas.commit()` — mid-execution state commit (Func `commit`)
- `reserve(amount, mode)` — balance reservation (Func `raw_reserve`)
- `tick()/tock()` — system actor lifecycle hooks (Func `run_ticktock`)
- `setCode()` — on-chain code upgrade (Func `set_code`)
- `crypto.*` — Ed25519 signatures, SHA-256, keccak-256
- `random.*` — on-chain pseudorandom number generation
- `config.param()` — read blockchain configuration parameters
- `bounce()` handler — explicit bounce recovery
- `try/catch` — exception handling (maps to TVM TRY)

4.5 Inheritance and Code Reuse

Tola supports familiar inheritance patterns via `abstract actor` and `is`:

```
abstract actor Ownable {
    ActorId owner;
    modifier onlyOwner() {
        require(msg.sender == owner, "not_owner");
        -;
    }
}

actor TokenBalance is Ownable {
    uint64 balance;
    function transfer(ActorId to, uint64 amount)
        external onlyOwner {
            require(balance >= amount);
            balance -= amount;
            to.send("credit", abi.encode(amount));
        }
}
```

4.6 Raw Cell Escape Hatch

When interoperating with existing FunC contracts or performing low-level operations, Tola provides `unsafe cell` blocks with full access to Cell/Slice/Builder primitives:

```
function handleLegacy(bytes rawBody) external {
    uint32 op;
    unsafe cell {
        slice s = rawBody.toSlice();
        op = s.loadUint(32);
    }
    if (op == 0x7362d09c) { processTransfer(); }
}
```

This ensures 100% FunC functionality coverage while keeping safe Tola as the default.

5 Built-in Standard Actor Library

5.1 Why Built-in Standard Actors

A decade of smart contract ecosystem development has demonstrated that standardized, audited building blocks are the single most impactful developer tool. The pattern is clear: 90% of on-chain applications are compositions of a few standard primitives (tokens, access control, governance, escrow).

The conventional approach—shipping libraries that each project copies, modifies, and deploys independently—creates fragmentation, redundant auditing, and thousands of subtly-different implementations of the same logic. TOS takes a fundamentally different approach: standard actors ship as **native C++ built-ins compiled into the node binary**:

	Copy-Deploy Model	TOS Built-in Actors
Code location	Copied into each project	One copy in node binary
Deployment cost	High (per-project bytecode)	Zero (create via message)
Audit burden	Each fork audited separately	Audited once, all benefit
Fragmentation	Thousands of variants	One canonical implementation
Upgrade	Each project independently	Node upgrade, all validators
Performance	VM bytecode interpretation	Native C++ execution

5.2 Standard Actor Inventory

Extracted from a decade of production smart contract ecosystem experience:

Token layer (ships with launch). TokenMaster, TokenBalance, TokenAllowance (fungible token standard); NFTCollection, NFTItem (non-fungible token standard); MultiToken (multi-asset standard).

Access control layer. Ownable, AccessControl (role-based), Multisig (multi-signature), Timelock (delayed execution).

Governance layer. Governor (proposal/voting/execution), VoteActor (voting power snapshots and delegation).

DeFi layer. AMMPool (constant-product swap), LiquidityPosition (concentrated liquidity), LendingPool (borrow/lend/liquidation), Escrow (custody/release), Vesting (token vesting schedules), PaymentSplitter (revenue distribution).

Infrastructure layer. Oracle (price feed interface), Proxy (upgradeable actor pattern).

5.3 Usage Model

Users write only the business logic that differentiates their application. Standard operations are handled by built-in actors:

```
actor Crowdfund {  
    ActorId tokenMaster;
```

```
ActorId treasury;
uint64 goal;
uint64 raised;

function contribute() external payable {
    raised += uint64(msg.value);
    tokenMaster.send("mint_to", abi.encode(
        ActorId.derive(tokenMaster, msg.sender),
        msg.value * 10
    ));
}

function finalize() external {
    require(raised >= goal, "goal_not_met");
    treasury.send("credit", abi.encode(raised));
}
}
```

In this example, the developer writes 15 lines of crowdfunding logic. Token minting, balance management, and transfer semantics are handled entirely by the built-in `TokenMaster` and `TokenBalance` actors.

6 AI Agent Economy

The rise of autonomous AI agents creates fundamentally new demands on blockchain infrastructure. By 2026, AI agents autonomously manage wallets, negotiate service payments, and execute thousands of micro-transactions per second. TOS's Actor Model is a natural fit:

6.1 Actors as Agent Wallets

Each AI agent maps to one or more actors. An agent's spending policy, authentication credentials, and transaction history are actor state— isolated, deterministic, and auditable. Unlike account-based chains where an agent must share state with a global contract, TOS actors give each agent its own execution context with guaranteed message ordering.

```
actor AgentWallet is Ownable {
    uint128 dailyLimit;
```

```
uint128 spentToday;
uint32 dayStart;

function pay(ActorId service, uint64 amount, bytes
memo) external {
    if (block.time > dayStart + 86400) {
        spentToday = 0;
        dayStart = block.time;
    }
    require(spentToday + amount <= dailyLimit, "
daily_limit");
    spentToday += amount;
    service.send("payment", abi.encode(self, amount
, memo),
        SendOptions { value: amount });
}
```

6.2 Throughput for Machine-to-Machine Economies

AI agents produce transaction volumes orders of magnitude higher than human users. Per-actor parallelism means thousands of agent wallets execute concurrently—each agent’s transactions are independent and never contend with other agents’ state. This is precisely the throughput model that agent-to-agent payment protocols (x402, MCP) require.

6.3 Verifiable Agent Behavior

Because actor state transitions are deterministic and recorded on-chain, any third party can verify an agent’s complete behavioral history. Combined with ZK proofs (Section 7), agents can prove they followed their programmed policy without revealing the policy itself—critical for competitive trading agents and confidential service agreements.

7 Privacy at the Actor Level

7.1 The Privacy Problem in Existing Blockchains

Current approaches to blockchain privacy fall into two categories: (1) privacy-specific chains (Zcash, Monero) that sacrifice programmability, and (2) privacy layers bolted onto transparent chains (Aztec on Ethereum, Tornado Cash) that are awkward, gas-expensive, and architecturally fragile.

The fundamental issue is that privacy in account-based systems requires encrypting the *entire* contract state or none of it. There is no natural boundary for partial confidentiality.

7.2 Actors as Natural Privacy Boundaries

TOS's Actor Model solves this by making each actor an independent privacy domain:

- Each actor's state is a separate HashmapE. Encrypting one actor's state entries does not affect any other actor.
- A **confidential actor** stores encrypted key-value pairs and attaches a zero-knowledge proof to each state transition, proving that the transition is valid without revealing the state itself.
- Other actors interact with the confidential actor through the same `.send()` interface—they see the message but not the internal state.
- Privacy is a *per-actor opt-in*, not a network mode. Public actors and confidential actors coexist in the same account, the same shard, the same block.

7.3 ZK-Verified Execution: Verify, Don't Re-Execute

The actor boundary also enables a shift from *re-execution* to *verification*:

1. The actor's owner (or a delegated prover) executes the transaction off-chain and produces a succinct ZK proof of correct state transition.
2. Validators receive the proof and the new state commitment (the HashmapE root hash).

3. Validators **verify the proof** (a constant-time operation) instead of re-executing the computation.

This is architecturally different from ZK rollups, which batch hundreds of transactions into one proof for a separate L1. In TOS, each *individual actor* can independently choose ZK-verified execution. The result: validators do less work as more actors adopt ZK proofs, and the network scales by *reducing per-transaction verification cost* rather than increasing hardware requirements.

A built-in `ZKVerifier` actor module will provide standard proof verification for common proof systems (Groth16, PLONK, STARK), so application developers can attach proofs without implementing verifiers from scratch.

8 Post-Quantum Resilience

8.1 The Threat

Shor’s algorithm on a sufficiently powerful quantum computer can break ECDSA and Ed25519—the signature schemes securing the vast majority of existing blockchains, including TOS’s current default—in polynomial time. Grover’s algorithm weakens symmetric primitives like SHA-256, though this is addressable by doubling key lengths. The existential threat is to *public-key cryptography*: any address whose public key has been exposed on-chain becomes immediately vulnerable the moment a cryptographically-relevant quantum computer is operational.

The industry consensus in 2026 is that this threat is 3–10 years away. NIST has already standardized post-quantum algorithms: CRYSTALS-Kyber (ML-KEM, FIPS 203) for key encapsulation, CRYSTALS-Dilithium (ML-DSA, FIPS 204) for digital signatures, and SPHINCS+ (SLH-DSA, FIPS 205) as a hash-based fallback.

8.2 Actor-Level Cryptographic Agility

Most blockchains face a dilemma: migrating to post-quantum signatures requires a network-wide hard fork, new address formats, and mass migration of user funds—an enormously disruptive operation.

TOS’s Actor Model provides a modular alternative:

- **Per-actor authentication.** Each actor can independently specify its signature scheme. A `PostQuantumAuth` built-in actor module replaces ECDSA/Ed25519 verification with SPHINCS+/SLH-DSA for any actor that opts in.
- **Gradual migration.** High-value actors (treasuries, bridges, governance) can adopt post-quantum signatures immediately. Low-value actors can migrate later. No chain-wide hard fork required.
- **Hybrid signatures.** During the transition period, actors can require *both* a classical Ed25519 signature and a post-quantum SLH-DSA signature, protecting against both current attacks and future quantum attacks simultaneously.
- **Algorithm rotation.** If a post-quantum scheme is later found to be weak, individual actors can rotate to a newer algorithm without affecting the rest of the network.

8.3 Hash-Based State Integrity

TOS’s Cell/HashmapE infrastructure is built on hash trees (SHA-256 or BLAKE2). Against Grover’s algorithm, these require only a doubling of hash output length (256-bit $\rightarrow$ 512-bit) to maintain equivalent security. This upgrade can be performed at the HashmapE level without changing the actor programming model—developers continue using `state_get/state_put` regardless of the underlying hash function.

9 Concrete Example: Parallel Token Transfers

Consider a token contract with millions of users. In the account model, every transfer serializes through one state tree:

```
UserA transfer --+
UserB transfer --+--> Token Account (serial)
UserC transfer --+
```

In TOS’s actor model, each user’s balance is an independent actor:

```
UserA transfer --> BalanceActor(hash(A)) --+
UserB transfer --> BalanceActor(hash(B)) --+--> all parallel
UserC transfer --> BalanceActor(hash(C)) --+
```

A transfer from Alice to Bob involves two actors:

1. `BalanceActor(Alice)` receives the transfer message, debits Alice's balance, and sends a `credit` message to `BalanceActor(Bob)`.
2. `BalanceActor(Bob)` receives the credit message and increments Bob's balance.

Because Alice's and Bob's actors have disjoint state, a simultaneous transfer from Charlie to Dave executes **fully in parallel** with no contention.

10 Infrastructure Stack

TOS's actor-oriented execution model is supported by a full infrastructure stack designed for high throughput, compact proofs, and decentralized networking:

- **Cell data structure** — all state and messages are encoded as directed acyclic graphs of Cells, providing native Merkle proofs at every level of the hierarchy.
- **BOC encoding** — a uniform Bag-of-Cells serialization format is used across blocks, network messages, and persistent storage.
- **Consensus** — Catchain/Simplex BFT for fast finality with Byzantine fault tolerance.
- **Networking** — ADNL (Abstract Datagram Network Layer), RLDP (Reliable Large Datagram Protocol), a Kademlia-based DHT, and QUIC transport.
- **Dynamic sharding** — address-prefix-based shard splitting allows the network to scale horizontally as load increases.
- **Storage** — CellDB backed by RocksDB for persistent state.

- **Execution phases** — storage → credit → compute → action → bounce, applied per actor transaction.
- **FunC/Tolk compatibility** — contracts written in FunC or Tolk run on TVM alongside Tola actors, sharing the same message protocol.
- **Lite-client / Tonlib** — lightweight client tools for querying chain state and submitting transactions.

The actor model determines the *granularity* of execution (the actor, not the account, is the scheduling unit) and the *developer interface* (key-value state rather than raw Cell/Slice/Builder manipulation). The infrastructure layers below this boundary are optimized for the actor workload from the start.

11 Comparison with Existing Platforms

	TOS	Serial-EVM Chains	Object/Resource Chains
Parallel execution	Actor-level	Planned / L2 only	Object/resource
Developer language	Tola (Solidity-like)	Solidity	New language (Move)
Learning curve	Low	Baseline	High
Reentrancy	Impossible	Major risk	N/A
Standard library	Native built-in	Copy-deploy per project	None standard
Hot-contract scaling	$M \times$ (per-actor)	$1 \times$	$M \times$ (per-object)
Privacy model	Per-actor ZK opt-in	L2 rollups only	None
Post-quantum	Per-actor migration	Global hard fork	Global hard fork
AI agent support	Native actor wallets	Add-on abstraction	None standard
ZK verification	Per-actor verify	L2 proof batching	None
Merkle proofs	Cell (built-in)	Trie-based	None

Existing platforms address parallelism, privacy, quantum resilience, and agent support as separate problems with separate, layered solutions—rollups for scaling, privacy protocols for confidentiality, hard forks for cryptographic migration, account abstraction for agents. TOS starts from first principles: the actor is the unit of execution, privacy, and cryptographic identity. Parallelism, confidentiality, and quantum resilience are not features added to the architecture— they are *consequences* of it.

12 Roadmap

1. **Phase 1: Actor State Isolation + Built-in Actors.** Add actor sub-structure to accounts, implement TVM opcodes (`STATEGET`/`STATESET`/`ACTORSEND`), build native Token/NFT actors, validate architecture end-to-end.
2. **Phase 2: Per-Actor Parallel Execution.** Each on-chain actor becomes a `td::actor::Actor`. Collator dispatches via `send_closure`. Different actors under the same account execute in parallel.
3. **Phase 3: Message Routing + Tola Compiler.** Add `actor_id` to `OutMsgQueue` key. Ship the Tola compiler (Solidity-like syntax $\rightarrow$ TVM bytecode). Full standard actor library (DeFi, governance, access control).
4. **Phase 4: Privacy Actors + ZK Verification.** Built-in `ConfidentialActor` and `ZKVerifier` modules. Per-actor opt-in encrypted state with ZK proof of correct state transition. Validators verify proofs instead of re-executing.
5. **Phase 5: Post-Quantum Authentication.** Built-in `PostQuantumAuth` module (SPHINCS+/SLH-DSA). Per-actor signature scheme migration. Hybrid classical+post-quantum mode for the transition period.
6. **Phase 6: AI Agent Economy.** Built-in `AgentWallet` with spending policies, daily limits, allowlists. Native integration with x402/MCP payment protocols. High-throughput agent-to-agent settlement.

Conclusion

Blockchains in 2026 face simultaneous demands for higher throughput, AI agent support, user privacy, and quantum resilience. Existing platforms address these as separate problems with separate solutions—rollups for scaling, privacy layers for confidentiality, hard forks for post-quantum migration, account abstraction for agents. The result is architectural complexity that grows with each new requirement.

TOS takes a different approach. The Actor Model is not merely a scaling technique—it is an *organizing principle* from which parallelism, privacy, quantum resilience, and agent support all emerge naturally:

CONCLUSION

- **Parallelism.** Hot contracts scale linearly with the number of independent actors—a direct consequence of state isolation.
- **Privacy.** Each actor is an independent privacy domain. Confidential actors encrypt their state and prove transitions via ZK proofs—a direct consequence of actor boundaries.
- **Quantum resilience.** Each actor independently adopts post-quantum signatures—a direct consequence of actor-level authentication.
- **AI agents.** Each agent is an actor with its own wallet, policy, and message interface—a direct consequence of the actor as the minimum execution unit.
- **Verification, not re-execution.** Validators verify ZK proofs instead of re-computing state transitions—a direct consequence of per-actor proof attachment.
- **Developer simplicity.** Solidity-like syntax, one new concept (`.send()` instead of `.call()`), and 90% of common functionality built-in and free to use.
- **Production-grade foundation.** Consensus, networking, storage, and Merkle proofs built on proven, well-understood techniques—Cell trees, BFT consensus, Kademlia DHT—chosen specifically to complement the actor execution model.

The key architectural insight is that Cell infrastructure need not be replaced—only *encapsulated*—and the actor boundary is not just a concurrency boundary but also a privacy boundary, a cryptographic boundary, and an economic boundary. One abstraction, many consequences.

References

- [1] G. AGHA, *Actors: A Model of Concurrent Computation in Distributed Systems*, MIT Press, 1986.
- [2] C. HEWITT, P. BISHOP, R. STEIGER, *A Universal Modular ACTOR Formalism for Artificial Intelligence*, IJCAI, 1973.
- [3] N. DUROV, *Telegram Open Network*, 2018.
- [4] G. WOOD, *A Secure Decentralised Generalised Transaction Ledger*, 2014 (updated 2024). Foundational work on serial-execution smart contract VMs.
- [5] S. BLACKSHEAR ET AL., *Move: A Language With Programmable Resources*, 2020. Resource-oriented approach to parallel smart contract execution.
- [6] M. CASTRO, B. LISKOV, *Practical Byzantine Fault Tolerance*, OSDI, 1999.
- [7] P. MAYMOUNKOV, D. MAZIÈRES, *Kademlia: A Peer-to-peer Information System Based on the XOR Metric*, IPTPS, 2002.
- [8] LINUX FOUNDATION, *x402: HTTP Payment Protocol for Agentic Commerce*, 2026. Adopted by Visa, Stripe, AWS, Anthropic, and Google Cloud for machine-to-machine payments.
- [9] ANTHROPIC, *Model Context Protocol (MCP): Standardizing Agent Tool Invocation*, 2025.
- [10] GOOGLE QUANTUM AI, *How a Quantum Computer Could Break Blockchain Cryptography*, reported in CoinDesk, April 2026.
- [11] D. BERNSTEIN ET AL., *SPHINCS+: Stateless Hash-Based Signatures*, NIST FIPS 205 (SLH-DSA), standardized August 2024.
- [12] L. DUCAS ET AL., *CRYSTALS-Dilithium: Digital Signatures from Module Lattices*, NIST FIPS 204 (ML-DSA), standardized August 2024.
- [13] J. GROTH, *On the Size of Pairing-Based Non-interactive Arguments*, EUROCRYPT, 2016.

REFERENCES

- [14] A. GABIZON, Z. WILLIAMSON, O. CIOBOTARU, *PLONK: Permutations over Lagrange-Bases for Oecumenical Noninteractive Arguments of Knowledge*, IACR ePrint 2019/953.
- [15] *Zero-Knowledge Rollups for Private Smart Contract Execution*, various implementations, 2023–2026.
- [16] *Account Abstraction for Programmable Wallets*, industry standard, 2023–2026.